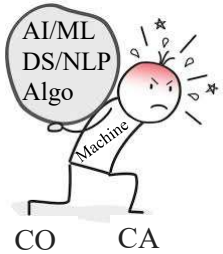
CS-2710-1 Spring 2026

Computer Organization and Systems (COS)



Lecture #10

25 February 2026

Ashoka University

BBB

IEEE 754 Single-Precision Floating-Point

Representation of floating point numbers in IEEE 754 standard:

single precision

	1	8	23
sign	S	E	M

FP-exponent E:
excess 127 binary coding
Actual exponent is $e = E - 127$

Floating-point exponent $E = e + 127$

mantissa:
sign + magnitude, normalized binary significand w/ hidden integer bit: 1.M

FP-exponent (*E*)	Actual exponent *e*
00000000 (0)	Special use
00000001 (1)	- 126
00000010 (2)	- 125
.....	.....
01111111 (127)	0
.....	.....
11111110 (254)	+ 127
11111111 (255)	Special use

How do I represent zero then ? → $1.M \times 2^{E-127}$

0/1	0000 0000	0000 0000 0000 0000 0000 000

FP-exponent (*E*)	Actual exponent *e*
00000000 (0)	Special use
00000001 (1)	- 126
00000010 (2)	- 125
.....	.....
01111111 (127)	0
.....	.....
11111110 (254)	+ 127
11111111 (255)	Special use

* Any non-zero number that is smaller than the smallest normalized FP-number is a denormal number (considering magnitude only)

Exponent	Mantissa	Object
0	0	Zero
0	Nonzero	Denormalized number*
1-254	Anything	+/- FP number
255	0	+ / - infinity
255	Nonzero	NaN like 0/0 or 0x inf

Single-Precision Normalized Range

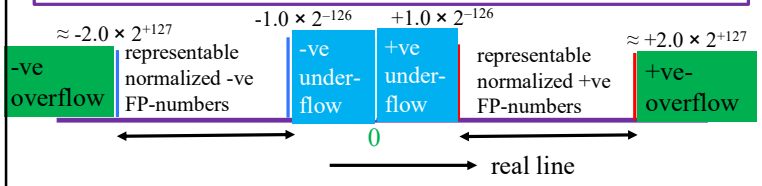
- Exponents 00000000 and 11111111 reserved
- Smallest value

0/1	0000 0001	0000 0000 0000 0000 0000 000

  - $\pm 1.0 \times 2^{-126} \approx \pm 1.2 \times 10^{-38}$
  - Largest value

0/1	1111 1110	1111 1111 1111 1111 1111 111

  - $= \pm(2.0 - 2^{-23}) \times 2^{+127} \approx \pm 3.4 \times 10^{+38}$



Single-Precision FP-Denormal Numbers

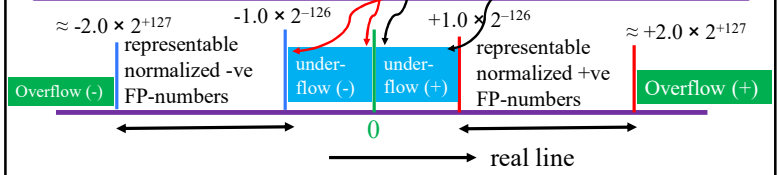
- Exponents 00000000 and 11111111 reserved
- Smallest value (normalized)

0/1	0000 0001	0000 0000 0000 0000 0000 000

  - $\pm 1.0 \times 2^{-126} \approx \pm 1.2 \times 10^{-38}$
  - Denormal numbers (gradual underflow)

0/1	0000 0000	M ≠ 0

$= \pm\, 0.M \times 2^{-126}$ {min: $\pm\, 2^{-23} \times 2^{-126}$; max: $\pm\, (1-2^{-23}) \times 2^{-126}$}



Denormal FP-Numbers

- Exponent = 000...0 ⇒ hidden bit is 0

$$N = (-1)^S \times (0.\text{Mantissa}) \times 2^{-126}$$

- Smaller than normalized numbers
  - allow for gradual underflow, with diminishing precision
- Denormal with mantissa = 000...0
- → +0, -0

Infinities and NaNs

- Exponent = 111...1, Mantissa= 000...0
  - ± Infinity
  - Can be used in subsequent calculations, avoiding need for overflow check
- Exponent = 111...1, Mantissa ≠ 000...0
  - Not-a-Number (NaN)
  - Indicates illegal or undefined result
    - e.g., 0.0 / 0.0, √-3
  - Can be used in subsequent calculations

Summary of IEEE 754 Encoding

Single-Precision	Exponent = 8	Fraction = 23	Value
Normalized Number	1 to 254	Anything	$\pm (1.F)_2 \times 2^{E-127}$
Denormalized Number	0	nonzero	$\pm (0.F)_2 \times 2^{-126}$
Zero	0	0	± 0
Infinity	255	0	± ∞
NaN	255	nonzero	NaN

Double-Precision	Exponent = 11	Fraction = 52	Value
Normalized Number	1 to 2046	Anything	$\pm (1.F)_2 \times 2^{E-1023}$
Denormalized Number	0	nonzero	$\pm (0.F)_2 \times 2^{-1022}$
Zero	0	0	± 0
Infinity	2047	0	± ∞
NaN	2047	nonzero	NaN

Overflow leads to ± ∞; underflow (gradual) → denormals; underflow (abrupt) → ± 0

Floating-Point Addition

- Consider a 3-digit mantissa binary example
  - $1.000_2 \times 2^{-1} + -1.111_2 \times 2^{-2}$
- 1. Align binary points
  - Shift right the number with smaller exponent
  - $1.000_2 \times 2^{-1} + -0.111_2 \times 2^{-1}$; rightmost 1 is lost
- 2. Add significands (integer addition)
  - $1.000_2 \times 2^{-1} + -0.111_2 \times 2^{-1} = 0.001_2 \times 2^{-1}$
- 3. Normalize result & check for over/underflow
  - $1.000_2 \times 2^{-4}$, with no over/underflow
- 4. Round and renormalize if necessary
  - $1.000_2 \times 2^{-4}$ (no change) = 0.0625

FP Adder Hardware

- Much more complex than integer adder
- Doing it in one clock cycle would take too long
  - Much longer than integer operations
  - Slower clock would penalize all instructions
- FP adder usually takes several cycles
  - Can be pipelined

Floating Point Addition Example

- Consider adding: $(+1.111)_2 \times 2^{-1} + (+1.011)_2 \times 2^{-3}$
  - For simplicity, we assume 4 bits of precision (or 3 bits of fraction)
- Cannot add significands … Why?
  - Because exponents are not equal
- How to make exponents equal?
  - Shift the significand of the lesser exponent right until its exponent matches the larger number (WHY shift lesser?)
- $(1.011)_2 \times 2^{-3} = (0.1011)_2 \times 2^{-2} = (0.01011)_2 \times 2^{-1}$
  - Difference between the two exponents = $-1 - (-3) = 2$
  - So, shift right by 2 bits
- Now, add the significands:

```
          1.111
        + 0.01011
        ---------
Carry →  10.00111
```

(as unsigned numbers) Max sum: 11.11…….110

Addition Example – cont'd

- So, $(1.111)_2 \times 2^{-1} + (1.011)_2 \times 2^{-3} = (10.00111)_2 \times 2^{-1}$
- However, result $(10.00111)_2 \times 2^{-1}$ is NOT normalized
- Normalize result: $(10.00111)_2 \times 2^{-1} = (1.000111)_2 \times 2^0$
  - In this example, we have a carry
  - So, shift right by 1 bit and increment the exponent
- Round the significand to fit in appropriate number of bits
  - We assumed 4 bits of precision or 3 bits of fraction
- Round to nearest: $(1.000111)_2 \approx (1.001)_2$
  - Renormalize if rounding generates a carry
- Detect overflow / underflow
  - If exponent becomes too large (overflow) or too small (underflow)

```
  1.000 | 111
+      1  ←┘
  -------
  1.001
```

Floating Point Subtraction Example

- Consider: $(1.000)_2 \times 2^{-3} + (-1.000)_2 \times 2^2$
  - We assume again: 4 bits of precision (or 3 bits of fraction)
- Shift significand of the lesser exponent right
  - Difference between the two exponents = 2 – (–3) = 5
  - Shift right by 5 bits: $(1.000)_2 \times 2^{-3} = (0.00001000)_2 \times 2^2$
- Convert subtraction into addition to 2's complement

Sign

2's Complement

```
  + 0.00001 × 2²
  – 1.00000 × 2²
  ──────────────
0 0.00001 × 2²
1 1.00000 × 2²
  ──────────────
1 1.00001 × 2²
```

2's Complement → `– 0.11111 × 2²`

Since result is negative, convert result from 2's complement to sign-magnitude

Subtraction Example – cont'd

- So, $(1.000)_2 \times 2^{-3} - (1.000)_2 \times 2^2 = -0.11111_2 \times 2^2$
- Normalize result: $-0.11111_2 \times 2^2 = -1.1111_2 \times 2^1$
  - For subtraction, we can have leading zeros
  - Count number *z* of leading zeros (in this case *z* = 1)
  - Shift left and decrement exponent by *z*
- Round the significand to fit in appropriate number of bits
  - We assumed 4 bits of precision or 3 bits of fraction
- Round to nearest: $(1.1111)_2 \approx (10.000)_2$
- Renormalize: rounding generated a carry

  $-1.1111_2 \times 2^1 \approx -10.000_2 \times 2^1 = -1.000_2 \times 2^2$
  - Result would have been accurate if more fraction bits are used

```
  1.111 1
+       1
  ────────
  10.000
```

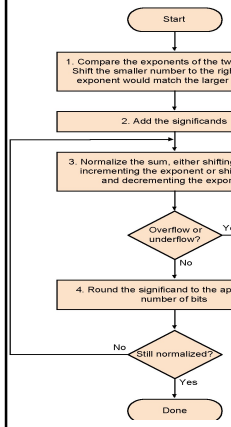
Floating-Point Addition: Salient Points

Can there be an overflow during the addition of significands?
1.111 . . 11 (maximum value of significand with restored leading 1)
1.111 . . 11 „
+ ----------
11.111 .. 10
When two numbers are both +ve/-ve, we add their magnitudes only; needs at most one extra bit to save the result; also, since there are no sign bits, no question of overflow in this step

How do you determine the sign of the result?
When signs are opposite, convert both numbers to 2'C, and then perform addition; the sign-bit of the sum determines the sign of the result; next reconvert it back to sign-magnitude form; no question of overflow in this step (WHY?)

Then, when overflow/underflow can occur during FP-addition?

Floating-Point Addition



If one number is +ve and the other is –ve, addition of significands should be done using 2's complement and the result is reconverted into sign-magnitude form

A + B =

0	1000 0001	0000 0000 0000 0000 0000 000

A + B = 10.0 × 2^1; A + B = 1.0 × 2^2

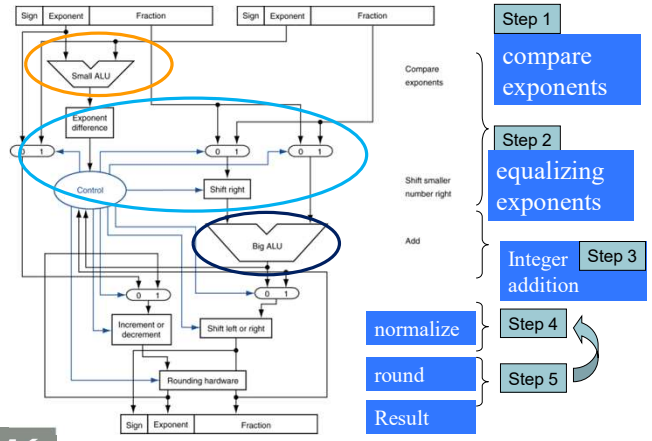
A: +0.1000 0000 0000 0000 0000 000 × 2^1

A: + 1.0000 0000 0000 0000 0000 000 × 2^0

B: + 1.1000 0000 0000 0000 0000 000 × 2^1

A	0	0111 1111	0000 0000 0000 0000 0000 000
B	0	1000 000	1000 0000 0000 0000 0000 000

FP Adder Hardware



Floating Point Multiplication Example

- Consider multiplying: $1.010_2 \times 2^{-1}$ by $-1.110_2 \times 2^{-2}$
  - As before, we assume 4 bits of precision (or 3 bits of fraction)
- Unlike addition, we add the exponents of the operands
  - Result exponent value = (–1) + (–2) = –3
- Using the biased representation: $E_Z = E_X + E_Y - Bias$
  - $E_X = (-1) + 127 = 126$ (*Bias* = 127 for SP)
  - $E_Y = (-2) + 127 = 125$
  - $E_Z = 126 + 125 - 127 = 124$ (value = –3)
- Now, multiply the significands:

$$\underbrace{(1.010)_2}_{\text{3-bit fraction}} \times \underbrace{(1.110)_2}_{\text{3-bit fraction}} = \underbrace{(10.001100)_2}_{\text{6-bit fraction}}$$

```
  1.010
× 1.110
 -------
    0000
   1010
  1010
 1010
 -------
10.001100
```

Multiplication Example – cont'd

- Since sign $S_X \neq S_Y$, sign of product $S_Z = 1$ (negative)
- So, $1.010_2 \times 2^{-1} \times -1.110_2 \times 2^{-2} = -10.001100_2 \times 2^{-3}$
- However, result: $-10.001100_2 \times 2^{-3}$ is NOT normalized
- Normalize: $10.001100_2 \times 2^{-3} = 1.0001100_2 \times 2^{-2}$
  - Shift right by 1 bit and increment the exponent
  - At most 1 bit can be shifted right … Why?
- Round the significand to nearest:

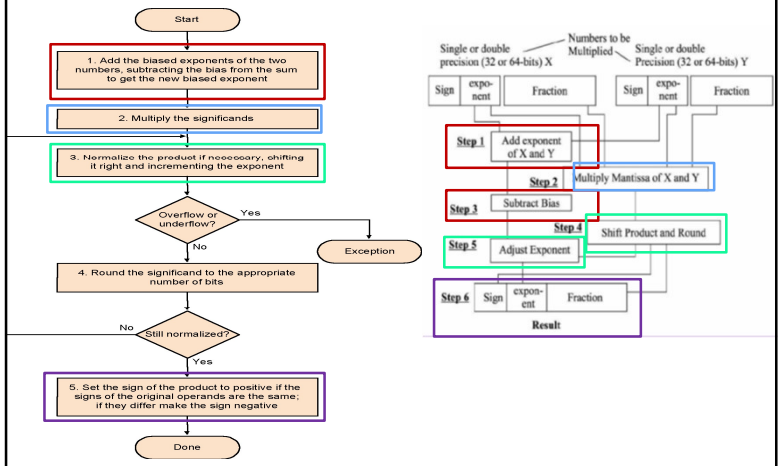
  $1.0001100_2 \approx 1.001_2$ (3-bit fraction)

  Result $\approx -1.001_2 \times 2^{-2}$ (normalized)

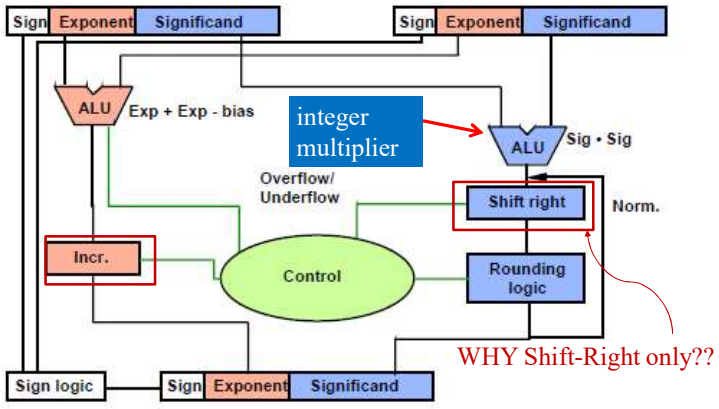
```
  1.000 1100
+       1
 ---------
  1.001
```

- Detect overflow / underflow
  - No overflow / underflow because exponent is within range

Floating Point Multiplication



FP Multiplier – Hardware Realization



FP Arithmetic Hardware

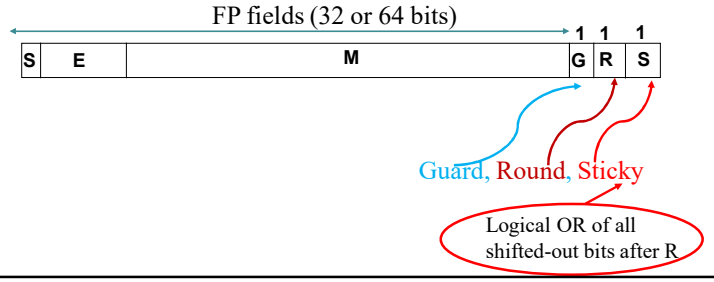
- FP multiplier is of similar complexity to FP adder
  - But uses a multiplier for significands instead of an adder
- FP arithmetic hardware usually does
  - Addition, subtraction, multiplication, division, reciprocal, square-root
  - FP ↔ integer conversion
- Operations usually takes several cycles
  - Can be pipelined

Extra Bits to Maintain Precision

- Floating-point numbers are approximations for …
  - Real numbers
- Infinite variety of real numbers exist between 1.0 and 2.0
  - However, exactly 2^{23} fractions can be represented in SP, and
  - Exactly 2^{52} fractions can be represented in DP (double precision)
- Extra bits are generated in intermediate results when …
  - Shifting and adding/subtracting a *p*-bit significand
  - Multiplying two *p*-bit significands (product can be 2*p* bits)
- But when packing result fraction, extra bits are discarded
- We only need few extra bits in an intermediate result
  - Minimizing hardware but without compromising precision

Three Extra Bits for Internal Use

- Three extra bits are added called Guard, Round, Sticky
  - To achieve accurate arithmetic such as rounding of significand, otherwise some bits would have been lost during right shifts
  - Reduces hardware without compromising precision



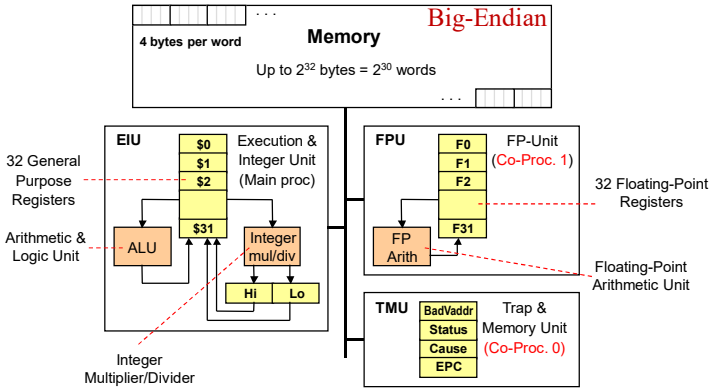
Guard Bit

- When we shift bits to the right for alignment, some bits are lost
- We may need to shift the result to the left for normalization after operation
- Storing the lost bits shifted to the right will make results more accurate during normalization
- Round and Sticky bits provide further handles for accurate rounding

MIPS Floating Point Coprocessor

- Called Coprocessor 1 or the Floating-Point Unit (FPU)
- 32 separate floating-point registers: $f0, $f1, …, $f31
- FP registers are 32 bits for single precision numbers
- Even-odd register pair form a double precision register
- Use the even number for double precision registers
  - $f0, $f2, $f4, …, $f30 are used for double precision
- Separate FP instructions for single/double precision
  - Single precision: add.s, sub.s, mul.s, div.s (.s extension)
  - Double precision: add.d, sub.d, mul.d, div.d (.d extension)
- FP instructions are more complex than the integer ones
  - Take more cycles to execute

The MIPS Processor: Full View



FP Instructions in MIPS

- FP hardware is Co-processor 1
  - Adjunct processor that extends the ISA
- Separate FP registers
  - 32 single-precision: $f0, $f1, … $f31
  - Paired for double-precision: $f0/$f1, $f2/$f3, …
    - Release 2 of MIPs ISA supports 32 × 64-bit FP reg's
- FP instructions operate only on FP registers
  - Programs generally don't do integer ops on FP data, or vice versa
  - More registers with minimal code-size impact
- FP load and store instructions
  - `lwc1`, `swc1`,
    - e.g., `lwc1 $f8, 32($sp)`

FP Instructions in MIPS

- Single-precision arithmetic
  - `add.s`, `sub.s`, `mul.s`, `div.s`
    - e.g., `add.s $f0, $f1, $f6`
- Double-precision arithmetic
  - `add.d`, `sub.d`, `mul.d`, `div.d`
    - e.g., `mul.d $f4, $f4, $f6`
- Single- and double-precision comparison
- Branch on FP condition code true or false

FP Arithmetic Instructions

Instruction	Meaning	Format					
add.s fd, fs, ft	(fd) = (fs) + (ft)	0x11	0	ft^5	fs^5	fd^5	0
add.d fd, fs, ft	(fd) = (fs) + (ft)	0x11	1	ft^5	fs^5	fd^5	0
sub.s fd, fs, ft	(fd) = (fs) – (ft)	0x11	0	ft^5	fs^5	fd^5	1
sub.d fd, fs, ft	(fd) = (fs) – (ft)	0x11	1	ft^5	fs^5	fd^5	1
mul.s fd, fs, ft	(fd) = (fs) × (ft)	0x11	0	ft^5	fs^5	fd^5	2
mul.d fd, fs, ft	(fd) = (fs) × (ft)	0x11	1	ft^5	fs^5	fd^5	2
div.s fd, fs, ft	(fd) = (fs) / (ft)	0x11	0	ft^5	fs^5	fd^5	3
div.d fd, fs, ft	(fd) = (fs) / (ft)	0x11	1	ft^5	fs^5	fd^5	3
sqrt.s fd, fs	(fd) = sqrt (fs)	0x11	0	0	fs^5	fd^5	4
sqrt.d fd, fs	(fd) = sqrt (fs)	0x11	1	0	fs^5	fd^5	4
abs.s fd, fs	(fd) = abs (fs)	0x11	0	0	fs^5	fd^5	5
abs.d fd, fs	(fd) = abs (fs)	0x11	1	0	fs^5	fd^5	5
neg.s fd, fs	(fd) = – (fs)	0x11	0	0	fs^5	fd^5	7
neg.d fd, fs	(fd) = – (fs)	0x11	1	0	fs^5	fd^5	7

FP Load/Store Instructions

- Separate floating point load/store instructions
  - lwc1: load word coprocessor 1
  - ldc1: load double coprocessor 1
  - swc1: store word coprocessor 1
  - sdc1: store double coprocessor 1

General purpose register is used as the base register

Instruction	Meaning	Format			
lwc1 $f2, 40($t0)	($f2) = Mem[($t0)+40]	0x31	$t0	$f2	im^{16} = 40
ldc1 $f2, 40($t0)	($f2) = Mem[($t0)+40]	0x35	$t0	$f2	im^{16} = 40
swc1 $f2, 40($t0)	Mem[($t0)+40] = ($f2)	0x39	$t0	$f2	im^{16} = 40
sdc1 $f2, 40($t0)	Mem[($t0)+40] = ($f2)	0x3d	$t0	$f2	im^{16} = 40

- Better names can be used for the above instructions
  - l.s = lwc1 (load FP single), l.d = ldc1 (load FP double)
  - s.s = swc1 (store FP single), s.d = sdc1 (store FP double)

FP Data Movement Instructions

- Moving data between general purpose and FP registers
  - mfc1: move from coprocessor 1 (to general purpose register)
  - mtc1: move to coprocessor 1 (from general purpose register)
- Moving data between FP registers
  - mov.s: move single precision float
  - mov.d: move double precision float = even/odd pair of registers

Instruction	Meaning	Format					
mfc1 $t0, $f2	($t0) = ($f2)	0x11	0	$t0	$f2	0	0
mtc1 $t0, $f2	($f2) = ($t0)	0x11	4	$t0	$f2	0	0
mov.s $f4, $f2	($f4) = ($f2)	0x11	0	0	$f2	$f4	6
mov.d $f4, $f2	($f4) = ($f2)	0x11	1	0	$f2	$f4	6

FP Convert Instructions

- Convert instruction: cvt.x.y
  - Convert to destination format x from source format y
- Supported formats
  - Single precision float = .s (single precision float in FP register)
  - Double precision float = .d (double float in even-odd FP register)
  - Signed integer word = .w (signed integer in FP register)

Instruction	Meaning	Format					
cvt.s.w fd, fs	to single from integer	0x11	0	0	fs^5	fd^5	0x20
cvt.s.d fd, fs	to single from double	0x11	1	0	fs^5	fd^5	0x20
cvt.d.w fd, fs	to double from integer	0x11	0	0	fs^5	fd^5	0x21
cvt.d.s fd, fs	to double from single	0x11	1	0	fs^5	fd^5	0x21
cvt.w.s fd, fs	to integer from single	0x11	0	0	fs^5	fd^5	0x24
cvt.w.d fd, fs	to integer from double	0x11	1	0	fs^5	fd^5	0x24

FP Compare and Branch Instructions

- FP unit (co-processor 1) has a condition flag
  - Set to 0 (false) or 1 (true) by any comparison instruction
- Three comparisons: equal, less than, less than or equal
- Two branch instructions based on the condition flag

Instruction	Meaning	Format					
c.eq.s fs, ft	cflag = ((fs) == (ft))	0x11	0	ft^5	fs^5	0	0x32
c.eq.d fs, ft	cflag = ((fs) == (ft))	0x11	1	ft^5	fs^5	0	0x32
c.lt.s fs, ft	cflag = ((fs) < (ft))	0x11	0	ft^5	fs^5	0	0x3c
c.lt.d fs, ft	cflag = ((fs) < (ft))	0x11	1	ft^5	fs^5	0	0x3c
c.le.s fs, ft	cflag = ((fs) <= (ft))	0x11	0	ft^5	fs^5	0	0x3e
c.le.d fs, ft	cflag = ((fs) <= (ft))	0x11	1	ft^5	fs^5	0	0x3e
bc1f Label	branch if (cflag == 0)	0x11	8	0	im^{16}		
bc1t Label	branch if (cflag == 1)	0x11	8	1	im^{16}		

Syscall Services

Service	$v0	Arguments / Result
Print Integer	1	$a0 = integer value to print
Print Float	2	$f12 = float value to print
Print Double	3	$f12 = double value to print
Print String	4	$a0 = address of null-terminated string
Read Integer	5	$v0 = integer read
Read Float	6	$f0 = float read
Read Double	7	$f0 = double read
Read String	8	$a0 = address of input buffer $a1 = maximum number of characters to read
Exit Program	10	
Print Char	11	$a0 = character to print
Read Char	12	$a0 = character read

Supported by MARS

FP Example: °F to °C

- C code:

```
float f2c (float fahr) {
  return ((5.0/9.0)*(fahr - 32.0));
}
```

  - fahr in $f12, result in $f0, literals in global memory space
- Compiled MIPS code:

```
f2c: lwc1  $f16, const5($gp)
     lwc2  $f18, const9($gp)
     div.s $f16, $f16, $f18
     lwc1  $f18, const32($gp)
     sub.s $f18, $f12, $f18
     mul.s $f0,  $f16, $f18
     jr    $ra
```

Integer Multiplication

Integer Multiplication: A Fundamental Question

What is the complexity of multiplying two n-bit integers?

$$
\begin{array}{rrrrrr}
 & & & A_3 & A_2 & A_1 & A_0 \\
 & & \times & B_3 & B_2 & B_1 & B_0 \\
\hline
 & & & A_3B_0 & A_2B_0 & A_1B_0 & A_0B_0 \\
 & & A_3B_1 & A_2B_1 & A_1B_1 & A_0B_1 & \\
 & A_3B_2 & A_2B_2 & A_1B_2 & A_0B_2 & & \\
+ & A_3B_3 & A_2B_3 & A_1B_3 & A_0B_3 & & \\
\hline
\end{array}
$$

AB_i called a "partial product"

A_i and B_i are all 0 or 1

Multiplying N-bit number by M-bit number gives (N+M)-bit result

Integer Multiplication

Multiplication
Given two n-bit unsigned integers a and b, compute $a \times b$

School method
$\Theta(n^2)$ bit operations

```
  1 1 0 1 0 1 0 1
× 0 1 1 1 1 1 0 1
-----------------
  1 1 0 1 0 1 0 1
0 0 0 0 0 0 0 0
1 1 0 1 0 1 0 1
1 1 0 1 0 1 0 1
1 1 0 1 0 1 0 1
1 1 0 1 0 1 0 1
1 1 0 1 0 1 0 1
0 0 0 0 0 0 0 0
-----------------
0 1 1 0 1 0 0 0 0 0 0 0 0 0 0 1
```

Ref: Jon Kleinberg and Éva Tardos, Algorithm Design

Copyright © 2005 Pearson-Addison Wesley

Q. Is school multiplication algorithm optimal?

Integer Multiplication

School method
$\Theta(n^2)$ bit operations

Multiplication
Given two n-bit unsigned integers a and b, compute $a \times b$

```
  1 1 0 1 0 1 0 1
× 0 1 1 1 1 1 0 1
-----------------
  1 1 0 1 0 1 0 1
0 0 0 0 0 0 0 0
1 1 0 1 0 1 0 1
1 1 0 1 0 1 0 1
1 1 0 1 0 1 0 1
1 1 0 1 0 1 0 1
1 1 0 1 0 1 0 1
0 0 0 0 0 0 0 0
-----------------
0 1 1 0 1 0 0 0 0 0 0 0 0 0 0 1
```

Andrey Nikolaevich Kolmogorov (1903 – 1987)

1952: Kolmogorov conjectured that any multiplication algorithm will take $\Omega(n^2)$ bit operations, i.e., it is asymptotically optimal

1960: Kolmogorov announced in a seminar at Moscow State University that it is indeed $\Omega(n^2)$

Q. Is grade-school multiplication algorithm optimal?

Integer Multiplication

School method
$\Theta(n^2)$ bit operations

Multiplication

Given two n-bit integers a and b, compute $a \times b$

```
  1 1 0 1 0 1 0 1
× 0 1 1 1 1 1 0 1
  1 1 0 1 0 1 0 1
0 0 0 0 0 0 0 0
1 1 0 1 0 1 0 1
1 1 0 1 0 1 0 1
1 1 0 1 0 1 0 1
1 1 0 1 0 1 0 1
1 1 0 1 0 1 0 1
0 0 0 0 0 0 0 0
0 1 1 0 1 0 0 0 0 0 0 0 0 0 0 1
```

Kolmogorov

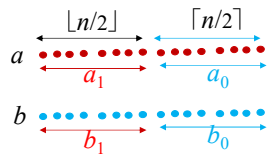
1960: Kolmogorov announced in a seminar at Moscow State University that multiplication is indeed $\Omega(n^2)$

Anatoly Alexeyevich Karatsuba (1937 – 2008)

1960: Within one week, Karatsuba disproved the claim, showing that multiplication can be done in $O(n^{1.585})$ time!

Karatsuba multiplication algorithm for two n-bit unsigned integers a, b

$$
\begin{aligned}
a &= 2^{n/2} \cdot a_1 + a_0 \\
b &= 2^{n/2} \cdot b_1 + b_0 \\
ab &= 2^n \cdot (a_1 b_1) + 2^{n/2} \cdot ((a_1 b_0) + (a_0 b_1)) + (a_0 b_0) \\
&= 2^n \cdot \underset{1}{a_1 b_1} + 2^{n/2} \cdot \underset{2}{((a_1 + a_0)(b_1 + b_0))} - \underset{1}{a_1 b_1} - \underset{3}{a_0 b_0}) + \underset{3}{a_0 b_0}
\end{aligned}
$$



To multiply two n-bit integers a and b:

- Add two $\frac{1}{2}n$ bit integers;
- Multiply three $\frac{1}{2}n$-bit integers, recursively;
- Add, subtract, and shift to obtain result;
- Four multiplications in each step of recurrence are reduced to three.

$$a_1 b_0 + a_0 b_1 = (a_1 + a_0)(b_1 + b_0) - a_1 b_1 - a_0 b_0$$

Theorem: [Karatsuba-Ofman 1962] One can multiply two n-bit integers in $O(n^{1.585})$ bit operations □

$$T(n) \le \underbrace{T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + T(1 + \lceil n/2 \rceil)}_{\text{recursive calls}} + \underbrace{\Theta(n)}_{\text{add, subtract, shift}} \Rightarrow T(n) = O(n^{\lg 3}) = O(n^{1.585})$$

Ref: Jon Kleinberg and Éva Tardos, Algorithm Design
Slides by Kevin Wayn
Copyright © 2005 Pearson-Addison Wesley

Multiplication of two n-bit integers a, b

Theorem: [Schönhage and Strassen] It is possible to multiply two n-bit integers in $O(n \log n \log \log n)$ bit operations

Ref: D. E. Knuth, The Art of Computer Programming, Vol. 2: Seminumerical Algorithms, Addison Wesley

---

Important Announcement

- Class-Test #1: Friday, 27 February 2026; in class;
- Open-book, open-notes; No electronics;
- Only handwritten sheets, printed material, and books allowed;
- Credit = 20%;
- Time: 10:10 AM – 11:40 AM;
- Coverage: Lecture #01 - Lecture #09;